

1 Report1

1.1 rp++

1.1.1 形式化定义

给定字符串 $S = s_1s_2\dots s_n$ ，其中 $n \leq 2 \times 10^6$ ，定义模式：

$$P = \text{rp}++ = [r][p][+][+]$$

要求统计 P 在 S 中的所有精确匹配次数

1.1.2 完整代码

这题水水的，唯一需要注意的是记得更新变量 `i += t.length()`，不然程序会死循环。

Listing 1: 题解 1

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      int cnt = 0;
6      string s, t = "rp++";
7      int len = t.length();
8      cin >> s;
9      string::size_type i = 0;
10     while((i = s.find(t, i)) != string::npos){
11         cnt++;
12         i += len;
13     }
14     printf("%d", cnt);
15     return 0;
16 }
```

1.2 数组游戏

1.2.1 形式化定义

给定初始空数组 $A_0 = \emptyset$ ，经过 q 次操作后得到数组序列 $\{A_t\}_{t=1}^q$ ，定义第 t 次操作后的炫酷值为：

$$F(A_t) = \sum_{i=1}^{|A_t|} A_t[i] \cdot i$$

操作类型集合 $\mathcal{O} = \{1, 2, 3\}$ 定义如下：

- op = 1: 循环右移, $A_t = \text{rotate}(A_{t-1}) = [a_n, a_1, \dots, a_{n-1}]$

- op = 2: 反转数组, $A_t = \text{reverse}(A_{t-1}) = [a_n, \dots, a_1]$
- op = 3 (k): 追加元素, $A_t = A_{t-1} \circ [k]$

对于复杂度:

- 操作次数: $q \leq 2 \times 10^5$
- 追加数值: $k \leq 10^6$
- 时间限制: 通常为 1-2 秒 (需 $O(1)$ 或 $O(\log n)$ 每次操作)

1.2.2 算法思路

本解法采用双端队列 + 标记法高效维护动态数组操作, 核心思想如下:

1. 状态维护:

- 使用双端队列 dq 存储实际数组元素
- 标记位 $reversed$ 记录当前是否处于反转状态
- 维护三个关键变量:

$$S = \sum_{i=1}^n (a_i \times i) \quad (\text{当前炫酷值})$$

$$T = \sum_{i=1}^n a_i \quad (\text{元素总和})$$

$$n = |dq| \quad (\text{数组长度})$$

2. 操作处理 (经数学推导可证明):

- 循环右移 (op=1):

因为:

$$S' = a_n \cdot 1 + \sum_{i=1}^{n-1} a_i \cdot (i+1) = S + \sum_{i=1}^n a_i - n \cdot a_n$$

所以有:

$$S \leftarrow S + \underbrace{T - n \times a_{\text{移动元素}}}_{\text{增量调整}}$$

根据 $reversed$ 状态决定移动首部或尾部元素

- 数组反转 (op=2):

因为:

$$S' = \sum_{i=1}^n a_{n-i+1} \cdot i = \sum_{j=1}^n a_j \cdot (n-j+1) = (n+1)T - S$$

所以有：

$$S \leftarrow (n+1) \times T - S$$

通过数学变换避免实际反转操作

- 追加元素 (op=3)：

显然有：

$$\begin{cases} S \leftarrow S + k \times (n+1) \\ T \leftarrow T + k \\ n \leftarrow n + 1 \end{cases}$$

根据 *reversed* 状态决定插入方向

1.2.3 复杂度分析

操作类型	时间复杂度
循环移位	$O(1)$
数组反转	$O(1)$
追加元素	$O(1)$
查询炫酷值	$O(1)$

1.2.4 关键技巧

- 数学变换：利用 $S_{\text{reverse}} = (n+1)T - S$ 的性质，避免实际反转操作
- 增量维护：通过分析操作前后的数学关系，仅更新受影响的部分
- 标记处理：用 *reversed* 标记统一处理正向/反向操作逻辑

1.2.5 完整代码

Listing 2: 题解 2

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int q;
6      cin >> q;
7      deque<int> dq;
8      long long S = 0, T = 0, n = 0;
9      bool reversed = false; // 标记是否反转
```

```

10
11     while (q--) {
12         int op;
13         cin >> op;
14         if (op == 1) { // 循环右移1位
15             if (!reversed) {
16                 int last = dq.back();
17                 dq.pop_back();
18                 dq.push_front(last);
19                 S += T - n * last;
20             } else {
21                 int first = dq.front();
22                 dq.pop_front();
23                 dq.push_back(first);
24                 S += T - n * first;
25             }
26         } else if (op == 2) { // 反转数组
27             reversed = !reversed;
28             S = (n + 1) * T - S;
29         } else if (op == 3) { // 追加元素k
30             int k;
31             cin >> k;
32             if (!reversed) {
33                 dq.push_back(k);
34             } else {
35                 dq.push_front(k);
36             }
37             n++;
38             T += k;
39             S += k * n;
40         }
41         printf("%lld\n", S);
42     }
43     return 0;
44 }

```

1.3 你喜欢 Inf、并查集和算法实验吗

1.3.1 形式化定义

给定社交关系图 $G = (V, E_f \cup E_h)$ ，其中：

- V 表示所有熟人节点 ($|V| = n$)
- E_f 为友谊边集 (无向边)
- E_h 为敌对边集 (无向边)
- 满足 $E_f \cap E_h = \emptyset$

一个合法派对 $P \subseteq V$ 需满足:

1. 朋友闭包性: $\forall v \in P$, 其所有朋友必须被邀请

$$N_f(v) \subseteq P \quad \text{其中} \quad N_f(v) = \{u \mid (v, u) \in E_f\}$$

2. 无敌对约束: 派对内不存在互相敌对的两人

$$\nexists (u, v) \in E_h \quad \text{s.t.} \quad u, v \in P$$

3. 连通性: 派对成员在友谊图中连通

$$\forall u, v \in P, \exists \text{路径 } u \leftrightarrow v \text{ 仅通过 } E_f \text{ 中的边}$$

1.3.2 输入输出要求

- 第一行: n (熟人数量, $2 \leq n \leq 2000$)
- 第二行: k (友谊边数, $0 \leq k \leq \min(10^5, \frac{n(n-1)}{2})$)
- 随后 k 行: 每行 $u_i \ v_i$ 表示一对朋友
- 下一行: m (敌对边数, $0 \leq m \leq \min(10^5, \frac{n(n-1)}{2})$)
- 随后 m 行: 每行 $u_j \ v_j$ 表示一对敌对关系

寻找满足条件的最大派对人数, 若不存在合法解则输出 0。

输入:

```

9
8
1 2
1 3
2 3
4 5
6 7
7 8
8 9
9 6
2
1 6
7 9

```

- 有效解: {1,2,3} (大小 3) 和 {4,5} (大小 2)
- 无效解: {6,7,8,9} (违反敌对约束), {1,2,3,4,5} (违反连通性)
- 输出: 3

1.3.3 并查集应用

我们注意到输入数据的特点, 每对人最多在输入中被提到一次, 两个人不能同时是朋友和彼此不喜欢, 所以我们可以考虑用并查集来管理不同的朋友链。

而在unite()的过程中, 需要注意维护当前朋友链的大小Size[n], 结合按秩合并的思想, 我们可以用这个数组来指导不同的朋友链之间的合并。

Listing 3: 题解 3: 并查集部分

```
1  int find(int x)
2  {
3      if (parent[x] != x)
4      {
5          parent[x] = find(parent[x]);
6      }
7      return parent[x];
8  }
9
10 void unite(int x, int y)
11 {
12     int rootX = find(x);
13     int rootY = find(y);
14     if (rootX != rootY)
15     {
16         if (Size[rootX] < Size[rootY])
17         {
18             swap(rootX, rootY);
19         }
20         parent[rootY] = rootX;
21         Size[rootX] += Size[rootY];
22     }
23 }
```

1.3.4 敌对关系检查

接下来我们进行敌对关系检查, 只需要逐一验证每个朋友链中是否存在敌对关系即可, 如果存在, 那么将这个朋友链标记为false即可。

Listing 4: 题解 3: 敌对关系检查部分

```

1  scanf("%d", &m);
2  for (int i = 1; i <= m; i++)
3  {
4      int u, v;
5      cin >> u >> v;
6      dislike[u].insert(v);
7      dislike[v].insert(u);
8  }
9
10 for (int i = 1; i <= n; i++)
11 {
12     int root = find(i);
13     if (!valid[root]) // 已经被否定的朋友链不再遍历
14         continue;
15     for (int j = 1; j <= n; j++)
16     {
17         if (find(j) == root) // 保证 j 与 i 在同一个朋友链中
18         {
19             for (int d : dislike[j])
20             {
21                 if (find(d) == root) // 如果 j 的敌对关系也在这个朋友链中，则否定该朋友链
22                 {
23                     valid[root] = false;
24                     break;
25                 }
26             }
27             if (!valid[root]) // 已经被否定的朋友链不再遍历
28                 break;
29         }
30     }
31 }

```

1.3.5 完整代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 2005;
5  int parent[MAXN];

```

```

6  int Size[MAXN]; // 不与size重复
7  bool valid[MAXN];
8  int n, k, m;
9  unordered_set<int> dislike[MAXN];
10 int find(int x)
11 {
12     if (parent[x] != x)
13     {
14         parent[x] = find(parent[x]);
15     }
16     return parent[x];
17 }
18
19 void unite(int x, int y)
20 {
21     int rootX = find(x);
22     int rootY = find(y);
23     if (rootX != rootY)
24     {
25         if (Size[rootX] < Size[rootY])
26         {
27             swap(rootX, rootY);
28         }
29         parent[rootY] = rootX;
30         Size[rootX] += Size[rootY];
31     }
32 }
33
34 int main()
35 {
36     // freopen("../in.txt", "r", stdin);
37     scanf("%d%d", &n, &k);
38
39     for (int i = 1; i <= n; i++)
40     {
41         parent[i] = i;
42         Size[i] = 1;
43         valid[i] = true;
44     }
45
46     for (int i = 1; i <= k; i++)

```



```

47     {
48         int u, v;
49         scanf("%d%d", &u, &v);
50         unite(u, v);
51     }
52
53     scanf("%d", &m);
54     for (int i = 1; i <= m; i++)
55     {
56         int u, v;
57         cin >> u >> v;
58         dislike[u].insert(v);
59         dislike[v].insert(u);
60     }
61
62     for (int i = 1; i <= n; i++)
63     {
64         int root = find(i);
65         if (!valid[root])
66             continue;
67         for (int j = 1; j <= n; j++)
68         {
69             if (find(j) == root)
70             {
71                 for (int d : dislike[j])
72                 {
73                     if (find(d) == root)
74                     {
75                         valid[root] = false;
76                         break;
77                     }
78                 }
79                 if (!valid[root])
80                     break;
81             }
82         }
83     }
84
85     int mx = 0;
86     for (int i = 1; i <= n; i++)
87     {

```

```

88     int root = find(i);
89     if (valid[root] && Size[root] > mx)
90     {
91         mx = Size[root];
92     }
93 }
94
95 printf("%d", mx);
96 return 0;
97 }

```

1.4 Sherway 的有向图

1.4.1 形式化定义

给定有向完全图 $G = (V, E)$ ，其中：

- 顶点集 $V = \{1, 2, \dots, n\}$
- 边集 E 由邻接矩阵 a 定义， $a_{i,j}$ 表示边 (i, j) 的权重
- 初始所有顶点处于关闭状态 $C = V$ ，开放顶点集 $O = \emptyset$

定义状态转移函数：

$$\begin{cases} O \leftarrow O \cup \{x\} & \text{操作 1 } x \\ O \leftarrow O \setminus \{x\} & \text{操作 3 (撤销最近操作 1)} \end{cases}$$

对于操作 2 $x \rightarrow y$ ，定义：

$$\delta(x, y) = \min_{\substack{p \in P(x, y) \\ p \cap C = \emptyset}} \sum_{(u, v) \in p} a_{u, v}$$

其中 $P(x, y)$ 为 x 到 y 的所有路径集合。

- 邻接矩阵满足 $a_{i,i} = 0, \forall i \in V$
- 操作约束：
 - 操作 1 前 $x \in C$
 - 操作 2 前 $x, y \in O$
 - 操作 3 前存在未撤销的操作 1
- 空间复杂度： $O(n^2)$ 存储邻接矩阵
- 时间复杂度：
 - 操作 1/3: $O(1)$
 - 操作 2: $O(n^2)$ (Floyd-Warshall 动态更新)

1.4.2 floyd 改进

题目的数据范围限定了用经典的复杂度为 $O(n^3)$ 的算法，会超时（只过两个点），我们要想到，大部分路线是没必要重新计算的，新引入一个点带来的影响的路径，**只可能是以这个点为中间结点的路径**。

Listing 5: 题解 4: floyd 改进部分

```
1 // 原本的floyd
2 void floyd()
3 {
4     for (int k = 1; k <= n; ++k)
5     {
6         if (!open[k])
7             continue;
8         for (int i = 1; i <= n; ++i)
9         {
10             if (!open[i])
11                 continue;
12             for (int j = 1; j <= n; ++j)
13             {
14                 if (!open[j])
15                     continue;
16                 if (cur[i][k] + cur[k][j] < cur[i][j])
17                 {
18                     cur[i][j] = cur[i][k] + cur[k][j];
19                 }
20             }
21         }
22     }
23 }
24
25 // 改进后的floyd
26 void floyd2(int x)
27 {
28     for (int i = 1; i <= n; ++i)
29     {
30         for (int j = 1; j <= n; ++j)
31         {
32             if (cur[i][j] > cur[i][x] + cur[x][j])
33             {
34                 cur[i][j] = cur[i][x] + cur[x][j];
35             }
36         }
37     }
```

```
37     }
38 }
```

1.4.3 操作 3 优化

因为操作 3 不是随机取消一个点，而是撤销上一个操作 1，因此我们可以[用一个栈来维护被操作 1 更新过的图](#)，这样我们就可以不再计算一次 floyd 算法，提高效率。

当然这里如果不采用 `cur` 数组来维护，全局只用一个栈，栈顶表示当前图也是可以的，这样在空间复杂度不变的情况下，避免了上面操作 3 的拷贝操作带来的时间损耗，可以提高运行效率。

Listing 6: 题解 4: 操作 3 优化部分

```
1  if (op == 1)
2  {
3      int x;
4      scanf("%d", &x);
5      open[x] = true;
6      op1s.push(x);
7
8      st_mp.push(vector<vector<int>>(n + 1, vector<int>(n + 1)));
9      for (int i = 1; i <= n; ++i)
10         for (int j = 1; j <= n; ++j)
11             st_mp.top()[i][j] = cur[i][j];
12     floyd2(x);
13 }
14 else if (op == 2)
15 {
16     int x, y;
17     scanf("%d_%d", &x, &y);
18     printf("%d\n", cur[x][y]);
19 }
20 else if (op == 3)
21 {
22     auto &backup = st_mp.top();
23     for (int i = 1; i <= n; ++i)
24         for (int j = 1; j <= n; ++j)
25             cur[i][j] = backup[i][j];
26     st_mp.pop();
27     open[op1s.top()] = false;
28     op1s.pop();
29 }
```

1.4.4 复杂度分析

操作类型	时间复杂度
打开结点	$O(n^2)$
查询最短路径	$O(1)$
撤销操作 1	$O(n^2)$ (可优化为 $O(1)$)

所以本题的总体时间复杂度为 $O(t_s \cdot n^2)$, 空间复杂度为 $O(t_1 \cdot n^2)$; 可以优化为时间复杂度为 $O(t_1 \cdot n^2)$, 空间复杂度为 $O(t_1 \cdot n^2)$ 。(t_s 是操作 1,3 的总数量, t_1 是操作 1 的数量)

1.4.5 完整代码

```
1  #include <bits/stdc++.h>
2  #define SHI_YAN_KE_JIE_SHU_LE 0
3  using namespace std;
4
5  const int INF = 1e9;
6  const int MAXN = 305;
7
8  int n, q;
9  int cur[MAXN][MAXN];
10 bool open[MAXN];
11 stack<int> op1s;
12 stack<vector<vector<int>>> st_mp;
13
14
15 void floyd2(int x)
16 {
17     for (int i = 1; i <= n; ++i)
18     {
19         for (int j = 1; j <= n; ++j)
20         {
21             if (cur[i][j] > cur[i][x] + cur[x][j])
22             {
23                 cur[i][j] = cur[i][x] + cur[x][j];
24             }
25         }
26     }
27 }
28
29 int main()
```

```

30 {
31     // freopen("in.txt", "r", stdin);
32     scanf("%d%d", &n, &q);
33     for (int i = 1; i <= n; ++i)
34     {
35         for (int j = 1; j <= n; ++j)
36         {
37             scanf("%d", &cur[i][j]);
38         }
39     }
40     memset(open, false, sizeof(open));
41     while (q--)
42     {
43         int op;
44         scanf("%d", &op);
45         if (op == 1)
46         {
47             int x;
48             scanf("%d", &x);
49             open[x] = true;
50             ops.push(x);
51
52             st_mp.push(vector<vector<int>>(n + 1, vector<int>(n + 1)));
53             for (int i = 1; i <= n; ++i)
54                 for (int j = 1; j <= n; ++j)
55                     st_mp.top()[i][j] = cur[i][j];
56             floyd2(x);
57         }
58         else if (op == 2)
59         {
60             int x, y;
61             scanf("%d%d", &x, &y);
62             printf("%d\n", cur[x][y]);
63         }
64         else if (op == 3)
65         {
66             auto &backup = st_mp.top();
67             for (int i = 1; i <= n; ++i)
68                 for (int j = 1; j <= n; ++j)
69                     cur[i][j] = backup[i][j];
70             st_mp.pop();

```

```
71         open[op1s.top()] = false;
72         op1s.pop();
73     }
74 }
75 return SHI_YAN_KE_JIE_SHU_LE;
76 }
```